

OSSP-PRACTICAL-WEEK2

Task-1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source code:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    printf("Source file opened successfully.\n");
    printf("Destination file opened successfully.\n");

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0) {
        perror("Error reading source file");
    } else {
        printf("File contents copied successfully.\n");
    }

    close(source);
    close(destination);

    printf("Files closed successfully.\n");

    return 0;
}
```

Output:

```
avulasanjana@SanjanaReddy:~$ nano week2pt2.c
avulasanjana@SanjanaReddy:~$ gcc week2pt2.c -o week2pt2
avulasanjana@SanjanaReddy:~$ echo "This is a sample file for system call demonstration." > source.txt
avulasanjana@SanjanaReddy:~$ cat source.txt
This is a sample file for system call demonstration.
avulasanjana@SanjanaReddy:~$ ./week2pt2
Source file opened successfully.
Destination file opened successfully.
File contents copied successfully.
Files closed successfully.
avulasanjana@SanjanaReddy:~$ |
```

Observation: I learned how Linux system calls such as `open()`, `read()`, `write()`, and `close()` allow a C program to interact with files. I also understood that these operations involve communication between user space and the Linux kernel, where the kernel provides the required file-handling services.

Task2: Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

Output:

```
avulasanjana@SanjanaReddy:~$ strace ./file
execve("./file", ["/file"], 0x7ffc0e6c04a8 /* 52 vars */) = 0
brk(NULL) = 0x55c1a1e2a000
arch_prctl(ARCH_SET_FS, 0x7f1b1f3f3740) = 0
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=212472, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 212472, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f1b1f39f000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1", 832) = 832
pread64(3, "\6\0\0\0\0\0\0\0", 8, 0x3c) = 8
pread64(3, "\3\0\0\0\0\0\0\0", 8, 0x18) = 8
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1922136, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\t\2\0\0\0\0\0\0", 8, 0x20) = 8
mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f1b1d5be000
mmap(0x7f1b1d5e4000, 1396736, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0x26000) = 0x7f1b1d5e4000
mmap(0x7f1b1d73a000, 339968, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x17c000) = 0x7f1b1d73a000
mmap(0x7f1b1d78d000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1cf000) = 0x7f1b1d78d000
mmap(0x7f1b1d793000, 53072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f1b1d793000
close(3) = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=41, ...}) = 0
read(3, "Hello, this is a sample file for", 1024) = 41
write(1, "Hello, this is a sample file for", 41)Hello, this is a sample file for = 41
read(3, "", 1024) = 0
close(3) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
avulasanjana@SanjanaReddy:~$
```

Observation: I learned how the strace utility can be used to observe the system calls made by a Linux command. I understood how commands such as cat interact with the Linux kernel through system calls to open, read, display, and close a file.